



الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة المعلوماتية

ضغط ملفات الصوت Audio Compression

تقرير مشروع مادة نظم الوسائط المتعددة

تسليم شادي الحكيم

آية عدنان عمران

مروة محمد زيواني

سارة عدنان عطية

إيمان نديم ياغي

فهرس المحتويات

3	مقدمة.....
3	بيئة التطبيق.....
3	الخوارزميات التي تم تطبيقها.....
5	الصيغ الصوتية المدعومة.....
5	الطلب الأول: إدخال الملف الصوتي وعرضه.....
7	الطلب الثاني: تشغيل وإيقاف الملف الصوتي.....
7	الطلب الثالث: عرض خصائص الملف الصوتي.....
8	الطلب السادس: إمكانية التحكم بإعدادات الضغط قبل تنفيذه.....
10	الطلب الرابع: ضغط الملفات الصوتية.....
16	الطلب الخامس: فك ضغط الملف الصوتي.....
19	الطلب السابع: مراقبة الأداء أثناء التنفيذ.....
19	الطلب الثامن: إلغاء عملية الضغط أثناء التنفيذ.....
20	الطلب التاسع: إعادة ضبط القيم الأصلية للملف الصوتي.....
20	الطلب العاشر: عرض تقرير بعد انتهاء عملية الضغط.....
22	الطلب الحادي عشر: حفظ الملف المضغوط والملف بعد الضغط.....
22	رابط المشروع على ال Git Hub.....

مقدمة

يهدف هذا المشروع إلى تطوير تطبيق سطح مكتب (Desktop Application) لضغط الملفات الصوتية باستخدام مجموعة من خوارزميات الضغط المختلفة، مع توفير إمكانية التحكم اليدوي في إعدادات الضغط وتخصيصها وفقاً لاحتياجات المستخدم. كما يتيح التطبيق مقارنة نتائج الخوارزميات المختلفة من حيث حجم الملف الناتج وجودة الصوت، بما يساهم في تحقيق التوازن بين كفاءة التخزين والمحافظة على جودة المحتوى الصوتي.

بيئة التطبيق

قمنا باعتماد لغة #c لتنفيذ التطبيق باستخدام نسخة 4.7.2 .NET Framework.

المكتبات المستخدمة:

1. NAudio

2. NAudio.Lame

3. TagLibSharp

4. OxyPlot

كما قمنا باستعمال FFmpeg لبعض عمليات فك الضغط، وهو إطار عمل مفتوح المصدر يُستخدم لمعالجة ملفات الوسائط المتعددة، مثل الصوت والفيديو. يوفر مجموعة كبيرة من الأدوات التي تسمح بتحويل الصيغ المختلفة، وضغط الملفات، وتعديلها، واستخراج البيانات منها.

الخوارزميات التي تم تطبيقها

1. خوارزمية Delta Modulation (DM)

تُعد خوارزمية Delta Modulation من أبسط خوارزميات ضغط الإشارات الصوتية. تعتمد فكرتها على تخزين اتجاه التغير بين العينات المتتالية بدلاً من تخزين القيمة الكاملة لكل عينة.

فبدلاً من استخدام عدة بتات لتمثيل قيمة العينة، يتم استخدام بت واحد فقط لكل عينة:

- القيمة (1) تعني أن الإشارة الحالية أكبر من القيمة المتوقعة ويجب زيادة التقدير.
- القيمة (0) تعني أن الإشارة الحالية أصغر من القيمة المتوقعة ويجب إنقاص التقدير.

وبذلك يتم تمثيل الإشارة كسلسلة من القرارات الثنائية التي تصف اتجاه تغيرها، مما يؤدي إلى تقليل كمية البيانات المطلوبة بشكل كبير. وبسبب ذلك، تُعد هذه الخوارزمية خوارزمية ضغط مع فقدان للبيانات (Lossy Compression)، لأنها تستبدل القيم الأصلية للعينات بمعلومات أبسط بكثير، مما يؤدي إلى فقدان جزء من التفاصيل الدقيقة للإشارة الصوتية أثناء عملية الضغط.

2. خوارزمية (ADM) Adaptive Delta Modulation :

تُعد خوارزمية ADM تطويرًا لخوارزمية DM، حيث تعتمد على تمثيل الإشارة من خلال تخزين اتجاه التغير بين العينات المتتالية بدلاً من تخزين القيمة الكاملة لكل عينة. تستخدم الخوارزمية بتًا واحدًا لكل عينة:

- القيمة 1 تعني أن الإشارة الحالية أكبر من القيمة المتوقعة.
- القيمة 0 تعني أن الإشارة الحالية أصغر من القيمة المتوقعة.

لكن بخلاف DM التقليدية، فإن حجم خطوة التغير (Δ Step Size) لا يكون ثابتًا، بل يتغير ديناميكيًا حسب طبيعة الإشارة:

- يتم زيادة الخطوة عندما تتغير الإشارة بسرعة.
- ويتم تقليل الخطوة عندما تكون الإشارة مستقرة أو تتغير ببطء.

يساعد ذلك على تحسين جودة الإشارة المعاد بناؤها وتقليل الأخطاء الناتجة عن استخدام خطوة ثابتة. وتُعد ADM خوارزمية ضغط مع فقدان للبيانات (Lossy Compression) لأنها لا تخزن القيم الأصلية للعينات وإنما تخزن معلومات مبسطة تصف تغيرها، مما يؤدي إلى تقليل حجم البيانات المطلوبة مع المحافظة على جودة أفضل مقارنةً بخوارزمية Delta Modulation التقليدية.

3. خوارزمية (DPCM) Differential Pulse Code Modulation :

تُعد خوارزمية DPCM تطويرًا لخوارزمية PCM، حيث تعتمد على تمثيل الإشارة من خلال تخزين الفرق بين العينة الحالية والعينة المتوقعة بدلاً من تخزين القيمة الكاملة لكل عينة. تعتمد الخوارزمية على وجود متنبئ (Predictor) يقوم بتقدير قيمة العينة الحالية اعتمادًا على العينات السابقة، ثم يتم حساب الفرق بين القيمة الحقيقية والقيمة المتوقعة:

- إذا كان الفرق موجبًا فهذا يعني أن العينة الحالية أكبر من القيمة المتوقعة.
- إذا كان الفرق سالبًا فهذا يعني أن العينة الحالية أصغر من القيمة المتوقعة.

وبدلاً من تخزين قيمة العينة الأصلية، يتم تخزين هذا الفرق بعد إجراء عملية التكميم (Quantization) عليه باستخدام عدد محدود من المستويات.

يساعد ذلك على تقليل كمية البيانات المطلوبة لأن الفروق بين العينات المتجاورة تكون عادةً أصغر من القيم الأصلية للعينات. وتُعد DPCM خوارزمية ضغط مع فقدان للبيانات (Lossy Compression) عند استخدام التكميم، لأنها لا تخزن القيم الأصلية بدقة كاملة وإنما تخزن نسخة مكتمة من الفروق، مما يؤدي إلى تقليل حجم البيانات مع المحافظة على جودة صوت جيدة مقارنةً بطرق الضغط البسيطة الأخرى.

الصيغ الصوتية المدعومة

- **WAV:** صيغة صوتية غير مضغوطة (Uncompressed Audio) ، تحتفظ بجميع البيانات الصوتية الأصلية دون فقدان أي جزء منها، مما يوفر أعلى جودة ممكنة على حساب حجم الملف الكبير.
- **MP3:** صيغة صوتية مضغوطة تعتمد على الضغط مع فقدان البيانات (Lossy Compression) ، حيث يتم تقليل حجم الملف بشكل كبير مع الحفاظ على جودة صوت مقبولة لمعظم الاستخدامات.
- **FLAC:** صيغة ضغط بدون فقدان للبيانات (Lossless Compression) ، تتيح تقليل حجم الملف مع الاحتفاظ الكامل بجودة الصوت الأصلية.
- **WMA:** اختصار لـ Windows Media Audio ، وهي صيغة صوتية طورتها شركة [Microsoft](#) تعتمد على تقنيات ضغط مختلفة وتتوفر بشكل أساسي على نظام التشغيل Windows ، لذلك يقتصر دعمها في التطبيق على بيئة ويندوز.
- **AIFF (AIF):** صيغة صوتية طورتها شركة [Apple](#) ، ونُسبته WAV من حيث كونها غالبًا غير مضغوطة وتوفر جودة صوت عالية، إلا أنها أكثر شيوعًا في أنظمة Apple .
- **AAC:** اختصار لـ Advanced Audio Coding ، وهي صيغة ضغط مع فقدان للبيانات (Lossy Compression) .
- **M4A:** حاوية صوتية تعتمد غالبًا على ترميز AAC داخليًا، وتتميز بتقديم جودة جيدة مع أحجام ملفات صغيرة نسبيًا.

الصيغ غير المدعومة

- لا يدعم التطبيق الصيغ التالية M3U و PLS و MIDI ، وذلك لأنها لا تمثل ملفات صوتية فعلية يمكن ضغطها أو إعادة ترميزها.
- **M3U و PLS:** عبارة عن ملفات قوائم تشغيل (Playlists) تحتوي على مراجع أو مسارات لملفات صوتية أخرى، ولا تحتوي على بيانات صوتية بحد ذاتها.
 - **MIDI:** لا يحتوي على تسجيل صوتي حقيقي، وإنما يخزن أوامر وتعليمات موسيقية تصف النغمات والآلات المستخدمة، لذلك يختلف جوهريًا عن الملفات الصوتية التقليدية.

الطلب الأول: إدخال الملف الصوتي وعرضه

1. إدخال الملف الصوتي:

قمنا بتحقيق إدخال الملف الصوتي بطريقتين وهما:

- عن طريق زر upload باستخدام OpenFileDialog
- عن طريق السحب والإفلات على Panel باستخدام الـ events :panelDropZone_DragEnter و panelDropZone_DragDrop

وفي كلتا الحالتين يتم تجهيز متغيرين وهما:

- `AudioFileReader audioFile`
يُستخدم لقراءة الملف الصوتي واستخراج البيانات الصوتية منه. يوفر إمكانية الوصول إلى العينات الصوتية (Samples) ومعلومات الملف مثل مدة التشغيل ومعدل العينات وعدد القنوات الصوتية.
- `WaveOutEvent outputDevice`
يُستخدم لتشغيل الصوت عبر أجهزة الإخراج الصوتي في نظام التشغيل، مثل السماعات أو سماعات الرأس. فهو يستقبل البيانات الصوتية من مصدر مثل `AudioFileReader` ويقوم بإرسالها إلى بطاقة الصوت لتشغيلها.

2. عرض الموجة الصوتية

يتم عرض الموجة الصوتية باستخدام الدالة `PlotWaveform` التي تقوم بقراءة الملف الصوتي وتحويل بياناته إلى تمثيل مرئي للموجة الصوتية باستخدام مكتبة `OxyPlot`. يتم تنفيذ العملية داخل `Thread` منفصل باستخدام `Task.Run` لتجنب تجميد واجهة المستخدم أثناء معالجة الملف.

3. مراحل الرسم:

- يتم فتح الملف باستخدام `AudioFileReader` واستخراج معدل العينات (Sample Rate) الخاص به.
- في حال كان الملف يحتوي على قناتين صوتيتين (Stereo)، يتم تحويله إلى قناة واحدة (Mono) لتبسيط عملية الرسم.
- يتم قراءة العينات الصوتية على دفعات وتخزينها في قائمة تمثل قيم سعة الإشارة (Amplitude).
- يتم إنشاء نموذج للرسم البياني وتخصيص مظهره من حيث الخلفية والمحاور وإخفاء التدريجات غير الضرورية.
- لتسريع عملية الرسم وتحسين الأداء، يتم اختيار عدد محدد من العينات بدلاً من عرض جميع العينات الموجودة في الملف.
- يتم تحويل كل عينة إلى نقطة على الرسم البياني بحيث يمثل المحور الأفقي الزمن بالثواني ويمثل المحور العمودي سعة الإشارة الصوتية.
- بعد الانتهاء من تجهيز البيانات، يتم تحديث عنصر الرسم في واجهة المستخدم وإظهار الشكل الموجي للملف الصوتي.

الطلب الثاني: تشغيل وإيقاف الملف الصوتي

تم توفير إمكانية التحكم في تشغيل الملف الصوتي من خلال استخدام مكتبة NAudio وباستخدام WaveOutEvent، حيث تم تنفيذ الوظائف الأساسية التالية:

- **تشغيل الملف الصوتي:** تم تحقيق ذلك باستخدام الدالة `Play()`، والتي تبدأ تشغيل الملف الصوتي المحمل.
 - **الإيقاف المؤقت (Pause):** تم توفير إمكانية إيقاف التشغيل مؤقتًا باستخدام الدالة `Pause()`، مما يسمح للمستخدم باستئناف التشغيل لاحقًا من نفس الموضع.
 - **الإيقاف الكامل (Stop):** تم تنفيذ الإيقاف الكامل باستخدام الدالة `Stop()`، مع إعادة مؤشر القراءة إلى بداية الملف بواسطة التعليمة `audioFile.Position = 0`، بحيث يبدأ التشغيل من البداية عند إعادة تشغيل الملف.
- كما تم دعم إمكانية الاستماع إلى الملف الصوتي قبل عملية الضغط للتحقق من محتواه وجودته، بالإضافة إلى إمكانية تشغيل الملف الناتج بعد الضغط مباشرة، حيث يتم تحميل الملف الجديد تلقائيًا وإتاحته للمستخدم للاستماع إليه ومقارنة النتائج.

الطلب الثالث: عرض خصائص الملف الصوتي

تم عرض المعلومات بالاعتماد على كل من `FileInfo` و `AudioFileReader` و `TagLib` للحصول على هذه الخصائص. الخصائص التي يتم عرضها هي:

- **حجم الملف (File Size):** تم الحصول عليه باستخدام `FileInfo.Length` ثم تحويله إلى وحدة MB.
- **نوع الملف (File Type):** تم استخراجه من لاحقة الملف باستخدام `FileInfo.Extension`.
- **مدة الملف (Duration):** تم الحصول عليها من الخاصية `TotalTime` التابعة للكائن `AudioFileReader`.
- **معدل العينات (Sample Rate):** تم استخراجه من `WaveFormat.SampleRate` وإظهاره بوحدة KHz.
- **عدد القنوات الصوتية (Channels):** تم الحصول عليه من `WaveFormat.Channels` لتحديد ما إذا كان الصوت أحادي القناة (Mono) أو ثنائي القناة (Stereo).
- **معدل البت (Bit Rate):** تم استخراجه باستخدام مكتبة `TagLib` من الخاصية `AudioBitrate`.
- **نوع الترميز (Encoding):** تم الحصول عليه من الخاصية `Description` في مكتبة `TagLib` والتي توضح طريقة ترميز الملف الصوتي.

الطلب السادس: إمكانية التحكم بإعدادات الضغط قبل تنفيذه

تم شرح هذا الطلب قبل الطلب الرابع والخامس بهدف تسهيل تسلسل الأفكار المتعلقة بآلية عمل التطبيق وتنفيذ عمليات الضغط. لتحقيق هذا المتطلب تم إنشاء نافذة (Form) مخصصة تتيح للمستخدم تحديد إعدادات الضغط قبل بدء العملية. ويتم تخزين جميع الإعدادات ضمن كائن من الصف CompressionSettings ليتم تمريرها إلى خوارزمية الضغط المختارة. وتم التمييز بين نوعين من الإعدادات:

1. الإعدادات العامة

وهي إعدادات يمكن تطبيقها على الملف الصوتي بغض النظر عن خوارزمية الضغط المستخدمة، وتشمل:

- **معدل العينات (Sample Rate):**
يحدد عدد العينات الصوتية المأخوذة في الثانية الواحدة. يؤدي تقليل هذه القيمة إلى تقليل حجم الملف الناتج على حساب جزء من الجودة الصوتية.
 - **عمق البت (Bit Depth):**
يحدد عدد البتات المستخدمة لتمثيل كل عينة صوتية. كلما زادت القيمة زادت دقة تمثيل الإشارة الصوتية وازداد حجم الملف.
 - **عدد القنوات (Channels):**
يحدد عدد القنوات الصوتية المستخدمة، مثل قناة واحدة (Mono) أو قناتين (Stereo). تقليل عدد القنوات يساهم في تقليل حجم الملف.
 - **معدل البت المستهدف (Target Bit Rate):**
يمثل معدل نقل البيانات الصوتية ويؤثر بشكل مباشر على حجم الملف النهائي وجودته.
- ومن المهم الإشارة إلى أن معدل البت (Bit Rate) لا يُعد إعداداً مستقلاً بالكامل، بل يتم حسابه فعلياً من خلال مجموعة من العوامل الأخرى مثل معدل العينات (Sample Rate) وعمق البت (Bit Depth) وعدد القنوات (Channels). لذلك فإن تغيير هذه القيم يؤدي تلقائياً إلى تغيير معدل البت الناتج.
- وكما يتم تحديد وتخزين خوارزمية الضغط المراد تطبيقها، وعند اختيارها تظهر مجموعة الإعداد الخاصة بكل خوارزمية

2. الإعدادات الخاصة بالخوارزميات

بالإضافة إلى الإعدادات العامة، عندما يتم اختيار خوارزمية معينة، تظهر إعدادات خاصة مرتبطة بطريقة عملها الداخلية.

• خوارزمية Delta Modulation

تعتمد هذه الخوارزمية على الإعداد التالي:

– حجم الخطوة (Step Size):

يمثل مقدار التغير المسموح به عند تتبع الإشارة الصوتية. تؤدي القيم الصغيرة إلى تمثيل أكثر دقة للتغيرات البسيطة في الإشارة، بينما تسمح القيم الكبيرة بتتبع التغيرات السريعة بشكل أفضل على حساب الدقة.

• خوارزمية Adaptive Delta Modulation (ADM / CVSD)

تعتمد هذه الخوارزمية على مجموعة من الإعدادات التي تتحكم بآلية تعديل حجم الخطوة (Step Size) بشكل ديناميكي أثناء عملية الضغط:

– Bit History Length:

يمثل عدد البتات السابقة التي يتم الاحتفاظ بها عند تحليل نمط تغير الإشارة. تستخدم هذه القيمة لتحديد ما إذا كانت الخوارزمية ستقوم بزيادة حجم الخطوة أو تقليله اعتماداً على اتجاه التغيرات الأخيرة.

– Min Step Size:

يمثل أصغر قيمة مسموحة لحجم الخطوة. يساعد ذلك على منع انخفاض قيمة الخطوة بشكل كبير مما قد يؤدي إلى بطء استجابة الخوارزمية للتغيرات في الإشارة.

– Max Step Size:

يمثل أكبر قيمة مسموحة لحجم الخطوة. يهدف هذا الحد إلى منع ازدياد الخطوة بشكل مبالغ فيه مما قد يسبب زيادة الخطأ وتشويه الإشارة المعاد بناؤها.

– Step Multiplier:

معامل يستخدم لزيادة حجم الخطوة عندما تكتشف الخوارزمية وجود تغيرات متتالية في الاتجاه نفسه، مما يسمح بتتبع التغيرات السريعة في الإشارة بشكل أفضل.

– Step Decay:

معامل يستخدم لتقليل حجم الخطوة عندما تصبح التغيرات في الإشارة أبطأ أو متذبذبة، مما يساعد على تحسين دقة تمثيل الإشارات الهادئة والتغيرات الصغيرة.

• خوارزمية Differential Pulse Code Modulation (DPCM)

تعتمد هذه الخوارزمية على الإعدادات التالية:

– Quantization Levels:

يمثل عدد مستويات التكميم المستخدمة عند تمثيل الفرق بين العينة الحالية والعينة المتوقعة. كلما ازداد عدد المستويات أصبح تمثيل الفروق أكثر دقة، مما يحسن جودة الإشارة المستعادة بعد فك الضغط ولكنه يؤدي إلى زيادة كمية البيانات المخزنة. أما استخدام عدد أقل من المستويات فيؤدي إلى زيادة نسبة الضغط على حساب دقة التمثيل وجودة الصوت الناتج.

تتحكم هذه القيمة بالتوازن بين جودة الصوت الناتج ومقدار الضغط الذي تحققه الخوارزمية، حيث تعتمد خوارزمية DPCM على ضغط فروق العينات بدلاً من تخزين قيم العينات الأصلية بشكل مباشر.

تم شرح تفاصيل هذه القيم الخاصة بالخوارزميات وتأثيرها على عمل الخوارزمية ضمن شرح الطلب الرابع الخاص بخوارزميات الضغط.

ولتخزين هذه الإعدادات تم إنشاء الصف `CompressionSettings` كحاوية موحدة لجميع إعدادات الضغط، بحيث يمكن مشاركتها وإعادة استخدامها من قبل جميع الخوارزميات المختلفة، مما يسهل إدارة الإعدادات ويضمن توحيد آلية تمريرها بين أجزاء النظام المختلفة.

الطلب الرابع: ضغط الملفات الصوتية

تم تحقيق ضغط الملفات الصوتية من خلال تطبيق ثلاث خوارزميات (DM, ADM, DPCM). ورغم اختلاف آلية عمل كل خوارزمية، فإن جميعها تتبع مجموعة من الخطوات العامة المشتركة.

أولاً: المعالجة المسبقة (Audio Preprocessing)

قبل تطبيق خوارزمية الضغط، يتم تمرير الملف الصوتي إلى الصف `AudioPreprocessor` المسؤول عن تنفيذ الإعدادات العامة التي حددها المستخدم مسبقاً. وتشمل هذه المرحلة ما يلي:

- **تعديل معدل العينات (Sample Rate) :**
في حال طلب المستخدم معدل عينات مختلف عن معدل الملف الأصلي، يتم إعادة أخذ العينات (Resampling) باستخدام `WdlResamplingSampleProvider`.
- **تعديل عدد القنوات (Channels) :**
يمكن تحويل الملف من Stereo إلى Mono أو العكس حسب الإعدادات المحددة.
- **تعديل عمق البت (Bit Depth) :**
يتم تنفيذ عملية إعادة التكميم (Quantization) لتقليل أو زيادة عدد المستويات المستخدمة لتمثيل العينات الصوتية. خلال عملية التكميم تمت إضافة كمية صغيرة من الضوضاء العشوائية (Dithering) قبل التكميم بهدف تقليل التشوهات الناتجة عن فقدان الدقة وتحسين جودة الإشارة بعد تقليل عمق البت.

بعد الانتهاء من هذه العمليات يتم إنشاء كائن من النوع `ProcessedAudio` يحتوي على:

- العينات الصوتية المعالجة (Samples)
- معدل العينات النهائي (Sample Rate)
- عمق البت النهائي (Bit Depth)
- عدد القنوات النهائي (Channels)

ويتم إعادة هذا الكائن إلى خوارزمية الضغط المختارة.

ثانياً: تطبيق خوارزمية الضغط

بعد تنفيذ المعالجة المسبقة، تقوم الخوارزمية المختارة بضغط العينات الصوتية وفق الآلية الخاصة بها. تختلف تفاصيل التنفيذ بين الخوارزميات المختلفة، إلا أن جميعها تعتمد على البيانات الناتجة من مرحلة المعالجة المسبقة.

ثالثاً: تخزين الملف المضغوط

بعد الانتهاء من عملية الضغط يتم إنشاء ملف بلاحقة مخصصة (Custom Extension) خاصة بالخوارزمية.

هذا الملف لا يمثل ملفاً صوتياً قابلاً للتشغيل مباشرة، وإنما يمثل حاوية لتخزين البيانات المضغوطة والمعلومات اللازمة لفك الضغط لاحقاً، ويتكون من جزأين رئيسيين:

1. Header: يحتوي على المعلومات الضرورية لإعادة بناء الملف الأصلي أثناء فك الضغط.

2. Compressed Data: يحتوي على البيانات الناتجة عن خوارزمية الضغط.

المعلومات العامة المخزنة في الـ Header :

تقوم جميع الخوارزميات بتخزين مجموعة مشتركة من المعلومات داخل الـ Header ، وتشمل:

- Sample Rate: معدل العينات المستخدم بعد المعالجة المسبقة.
- Bit Depth: عمق البت المستخدم لتمثيل العينات.
- Channels: عدد القنوات الصوتية.
- Bit Rate: معدل البت الناتج للملف المضغوط.
- Original Format: الصيغة الأصلية للملف قبل الضغط.
- Original Bit Rate: معدل البت الخاص بالملف الأصلي قبل تطبيق الضغط.

تُستخدم هذه المعلومات أثناء عملية فك الضغط لإعادة إنشاء الملف الصوتي بالشكل الصحيح.

أما المعلومات الإضافية الخاصة بكل خوارزمية، فتختلف باختلاف طريقة عمل الخوارزمية، وسيتم شرحها بشكل منفصل ضمن شرح كل خوارزمية من خوارزميات الضغط المستخدمة في المشروع.

شرح تطبيق ضغط الخوارزميات المطبقة:

1. خوارزمية Delta Modulation (DM)

– تطبيق الخوارزمية في المشروع:

عند بدء عملية الضغط يتم أولاً تطبيق مرحلة المعالجة المسبقة (Preprocessing) للحصول على العينات الصوتية بعد تنفيذ الإعدادات العامة التي اختارها المستخدم مثل معدل العينات وعمق البت وعدد القنوات.

بعد ذلك يتم حساب قيمة Step Size المستخدمة في الخوارزمية. ونظراً لأن هذه القيمة تؤثر بشكل مباشر على جودة الإشارة المضغوطة، لم يتم الاعتماد على قيمة ثابتة لجميع الملفات الصوتية. فالملفات ذات الإشارات الهادئة تحتاج إلى خطوة صغيرة للحصول على دقة جيدة، بينما تحتاج الملفات ذات الإشارات القوية أو المتغيرة بسرعة إلى خطوة أكبر لتجنب فقدان التغيرات السريعة في الإشارة.

لذلك تم الاعتماد على قيمة RMS (Root Mean Square) لحساب قيمة Step Size بشكل تلقائي اعتماداً على خصائص الملف الصوتي. تمثل RMS مقياساً لمتوسط شدة أو طاقة الإشارة الصوتية، ويتم حسابها من خلال إيجاد متوسط مربعات قيم العينات ثم أخذ الجذر التربيعي للنتيجة. وبذلك نحصل على قيمة تعبر عن المستوى العام للإشارة بغض النظر عن كون العينات موجبة أو سالبة.

بعد حساب RMS لجميع العينات يتم تحديد قيمة Step Size كنسبة من هذه القيمة، مع تطبيق حد أدنى وحد أقصى لمنع الحصول على قيم صغيرة جداً أو كبيرة جداً. يتيح ذلك للخوارزمية التكيف تلقائياً مع طبيعة الملف الصوتي وتحقيق أداء أفضل مقارنة باستخدام قيمة ثابتة لجميع الملفات. بعد تجهيز البيانات تبدأ عملية الترميز وفق الخطوات التالية:

1. يتم إنشاء متغير يمثل القيمة المتوقعة الحالية للإشارة (Predicted Sample) لكل قناة صوتية.
 2. تتم مقارنة كل عينة صوتية بالقيمة المتوقعة الحالية.
 3. إذا كانت العينة أكبر من القيمة المتوقعة يتم تخزين البت (1) وزيادة القيمة المتوقعة بمقدار Step Size.
 4. إذا كانت العينة أصغر من القيمة المتوقعة يتم تخزين البت (0) وإنقاص القيمة المتوقعة بالمقدار نفسه.
 5. يتم تقييد القيمة المتوقعة ضمن المجال $[-1, 1]$ للحفاظ على صحة الإشارة.
 6. يتم تجميع البتات الناتجة داخل مصفوفة بايتات بحيث يحتوي كل بايت على 8 عينات مضغوطة.
- وبذلك يتم تمثيل الملف الصوتي كاملاً كسلسلة من البتات التي تصف اتجاه تغير الإشارة فقط بدلاً من تخزين قيم العينات الأصلية.

البيانات المخزنة في Header :

بالإضافة إلى المعلومات العامة المشتركة بين جميع الخوارزميات، تقوم خوارزمية Delta Modulation بتخزين المعلومات التالية:

- SampleCount: عدد العينات الصوتية الموجودة في الملف.
 - StepSize: قيمة الخطوة المستخدمة أثناء عملية الترميز.
- تستخدم هذه المعلومات لاحقاً أثناء عملية فك الضغط لإعادة بناء الإشارة الصوتية بالشكل الصحيح.

معدل البت الناتج:

بما أن الخوارزمية تستخدم بتاً واحداً فقط لكل عينة ولكل قناة صوتية، فقد تم حساب معدل البت الناتج وفق العلاقة التالية:

$$\text{BitRate} = \text{SampleRate} \times \text{Channels}$$

أي أن معدل البت يعتمد مباشرة على معدل العينات وعدد القنوات المستخدمة بعد مرحلة المعالجة المسبقة.

– حساب معدل البت الأصلي:

تم استخراج معدل البت الخاص بالملف الأصلي من خلال حساب عدد البتات الموجودة في الملف مقسوماً على مدة التسجيل الصوتي، وذلك بهدف تخزين هذه المعلومة داخل الـ Header واستخدامها لأغراض المقارنة وعرض الإحصائيات للمستخدم.

2. خوارزمية Adaptive Delta Modulation (ADM)

– تطبيق الخوارزمية في المشروع:

عند بدء عملية الضغط يتم أولاً تطبيق مرحلة المعالجة المسبقة (Preprocessing) للحصول على العينات الصوتية بعد تنفيذ الإعدادات العامة التي اختارها المستخدم مثل معدل العينات وعمق البت وعدد القنوات.

بعد ذلك يتم حساب قيمة Initial Step Size الابتدائية التي ستستخدمها الخوارزمية في بداية عملية الترميز. وكما في خوارزمية Delta Modulation، تم الاعتماد على قيمة RMS للإشارة الصوتية للحصول على قيمة ابتدائية تتناسب مع طبيعة الملف، حيث يتم حساب RMS لجميع العينات ثم اختيار قيمة ابتدائية للخطوة كنسبة من هذه القيمة مع تطبيق حدود دنيا وعليا (MinStepSize , MaxStepSize) لضمان بقائها ضمن مجال مناسب. بعد تجهيز البيانات تبدأ عملية الترميز وفق الخطوات التالية:

1. يتم إنشاء قيمة متوقعة للإشارة (Predicted Sample) لكل قناة صوتية.
2. يتم إنشاء قيمة Step Size مستقلة لكل قناة وتبدأ بالقيمة الابتدائية المحسوبة.
3. تتم مقارنة كل عينة صوتية بالقيمة المتوقعة الحالية.
4. إذا كانت العينة أكبر من القيمة المتوقعة يتم تخزين البت (1) وزيادة القيمة المتوقعة بمقدار Step Size الحالية.
5. إذا كانت العينة أصغر من القيمة المتوقعة يتم تخزين البت (0) وإنقاص القيمة المتوقعة بالمقدار نفسه.
6. يتم تقييد القيمة المتوقعة ضمن المجال [-1,1] للحفاظ على صحة الإشارة.
7. يتم تخزين البت الناتج ضمن مصفوفة البايتات المضغوطة.
8. يتم الاحتفاظ بتاريخ لأحدث البتات المولدة لكل قناة صوتية باستخدام قائمة (Bit History).
9. إذا كانت جميع البتات الموجودة في السجل تحمل القيمة نفسها، فهذا يدل على أن الإشارة تتغير باستمرار في الاتجاه نفسه، وعندها يتم زيادة Step Size بضربها بالمعامل StepMultiplier.
10. أما إذا كانت البتات متغيرة بين 0 و1، فهذا يعني أن الإشارة مستقرة نسبياً أو تتذبذب حول قيمة معينة، ولذلك يتم تقليل Step Size باستخدام المعامل StepDecay.

11. بعد كل تعديل يتم التأكد من بقاء Step Size ضمن المجال المحدد بين MinStepSize و MaxStepSize.

وبذلك تستطيع الخوارزمية تعديل حجم الخطوة بشكل ديناميكي أثناء الترميز، مما يسمح لها بتتبع التغيرات السريعة في الإشارة بكفاءة أعلى مقارنة بخوارزمية Delta Modulation التقليدية ذات الخطوة الثابتة.

– البيانات المخزنة في Header :

بالإضافة إلى المعلومات العامة المشتركة بين جميع الخوارزميات، تقوم خوارزمية Adaptive Delta Modulation بتخزين المعلومات التالية:

- SampleCount : عدد العينات الصوتية الموجودة في الملف.
- InitialStepSize : قيمة الخطوة الابتدائية المستخدمة عند بدء الترميز.
- MinStepSize : أصغر قيمة مسموحة لحجم الخطوة.
- MaxStepSize : أكبر قيمة مسموحة لحجم الخطوة.
- StepMultiplier : معامل زيادة حجم الخطوة عند اكتشاف تغيرات متتالية في الاتجاه نفسه.
- StepDecay : معامل تقليل حجم الخطوة عند انخفاض الحاجة إلى قيم كبيرة.
- BitHistoryLength : عدد البتات المستخدمة لتحليل سلوك الإشارة واتخاذ قرار زيادة أو تقليل حجم الخطوة.

تُستخدم هذه المعلومات أثناء فك الضغط لإعادة تنفيذ آلية التكيف نفسها والحصول على الإشارة المعاد بناؤها بشكل صحيح.

– معدل البت الناتج:

بما أن خوارزمية Adaptive Delta Modulation تستخدم أيضاً بتاً واحداً فقط لكل عينة ولكل قناة صوتية، فإن معدل البت الناتج يُحسب وفق العلاقة:

$$\text{BitRate} = \text{SampleRate} \times \text{Channels}$$

أي أن معدل البت يعتمد مباشرة على معدل العينات وعدد القنوات الناتجين عن مرحلة المعالجة المسبقة.

– حساب معدل البت الأصلي:

يتم حسابه بنفس الطريقة في خوارزمية DM.

3. خوارزمية (DPCM) Differential Pulse Code Modulation

– تطبيق الخوارزمية في المشروع:

عند بدء عملية الضغط يتم أولاً تطبيق مرحلة المعالجة المسبقة (Preprocessing) للحصول على العينات الصوتية بعد تنفيذ الإعدادات العامة التي اختارها المستخدم مثل معدل العينات وعمق البت وعدد القنوات.

بعد ذلك يتم فصل العينات الصوتية بحسب القنوات (Deinterleaving)، بحيث يتم إنشاء مصفوفة مستقلة لكل قناة صوتية ومعالجة كل قناة بشكل منفصل.

تعتمد خوارزمية DPCM على فكرة أن الفرق بين العينات المتجاورة يكون غالباً أصغر بكثير من قيم العينات نفسها،

لذلك بدلاً من تخزين القيم الأصلية للعينات يتم تخزين الفروق بينها فقط.

تبدأ عملية الترميز وفق الخطوات التالية:

1. يتم حفظ أول عينة من كل قناة صوتية كما هي لأنها تمثل نقطة البداية اللازمة لإعادة بناء الإشارة لاحقاً.
 2. يتم حساب الفرق بين كل عينة والعينة السابقة لها لكل قناة صوتية.
 3. يتم إيجاد أصغر وأكبر قيمة للفروق الناتجة ضمن كل قناة.
 4. يتم تطبيع (Normalize) قيم الفروق إلى المجال [0,1] اعتماداً على أصغر وأكبر فرق تم إيجادهما.
 5. يتم تكميم (Quantization) الفروق باستخدام عدد مستويات التكميم المحدد ضمن الإعدادات Quantization Levels.
 6. يتم تحويل كل قيمة إلى رقم صحيح يمثل مستوى التكميم الموافق لها.
 7. بعد الانتهاء من تكميم جميع الفروق يتم تجميع القيم الناتجة داخل البايتات باستخدام تقنية Bit Packing ، بحيث يتم استخدام أقل عدد ممكن من البتات لتمثيل كل قيمة وفق عدد مستويات التكميم المختارة.
- وبذلك لا يتم تخزين الإشارة الأصلية بشكل مباشر، وإنما يتم تخزين أول عينة من كل قناة بالإضافة إلى الفروق المكممة بين العينات المتتالية، مما يؤدي إلى تقليل كمية البيانات المطلوبة مقارنة بتخزين العينات الأصلية كاملة.

– البيانات المخزنة في Header:

بالإضافة إلى المعلومات العامة المشتركة بين جميع الخوارزميات، تقوم خوارزمية DPCM بتخزين المعلومات التالية:

- SampleCount: عدد العينات الموجودة في كل قناة صوتية.
- QuantizationLevels: عدد مستويات التكميم المستخدمة أثناء ترميز الفروق.
- MinDifference: أصغر قيمة فرق تم إيجادها أثناء عملية الضغط.
- MaxDifference: أكبر قيمة فرق تم إيجادها أثناء عملية الضغط.
- InitialSamples: أول عينة في كل قناة صوتية، وتستخدم كنقطة بداية أثناء إعادة بناء الإشارة عند فك الضغط.

تُستخدم هذه المعلومات لاحقاً أثناء عملية فك الضغط لإعادة إنشاء الفروق المكممة ثم إعادة بناء الإشارة الصوتية.

– معدل البت الناتج:

على عكس خوارزميات DM و ADM ، لا يكون معدل البت ثابتاً في خوارزمية DPCM ، بل يعتمد على عدد مستويات التكميم المستخدمة، يتم أولاً حساب عدد البتات اللازمة لتمثيل كل مستوى تكميم وفق العلاقة:

$$\text{BitsPerValue} = \lceil \log_2(\text{QuantizationLevels}) \rceil$$

وبالتالي يتم تخصيص هذا العدد من البتات لكل فرق بعد عملية التكميم. لذلك كلما انخفض عدد مستويات التكميم انخفض حجم الملف الناتج وازدادت نسبة الضغط، بينما يؤدي استخدام عدد أكبر من المستويات إلى تحسين جودة الإشارة المستعادة على حساب زيادة حجم الملف.

– حساب معدل البت الأصلي:

تم استخراج معدل البت الخاص بالملف الأصلي باستخدام مكتبة TagLib من خصائص الملف الصوتي، ثم تم تخزينه ضمن الـ Header بهدف استخدامه لاحقاً في التقارير والإحصائيات ومقارنة نتائج الضغط.

الطلب الخامس: فك ضغط الملف الصوتي

تم تنفيذ عملية فك الضغط بحيث تتبع جميع الخوارزميات الخطوات العامة نفسها، مع اختلاف آلية فك الترميز الداخلية لكل خوارزمية بحسب طريقة الضغط المستخدمة، والخطوات هي:

1. قراءة ملف الضغط المخصص الذي تم إنشاؤه أثناء عملية الضغط. يحتوي هذا الملف على قسم Header حيث يحتوي على جميع المعلومات اللازمة لإعادة بناء الملف الصوتي.
بعد قراءة هذه المعلومات يتم تطبيق عملية فك الضغط باستخدام القيم المخزنة داخل الـ Header وبشكل معاكس تماماً للخطوات التي تم تنفيذها أثناء الضغط، مما يؤدي إلى استعادة العينات الصوتية الأصلية أو أقرب تمثيل ممكن لها.
2. بعد الانتهاء من فك الضغط يتم إنشاء ملف بصيغة WAV كمرحلة وسيطة. تم اختيار WAV لأنه يمثل صيغة صوتية غير مضغوطة، مما يجعله مناسباً كنقطة انطلاق للتحويل إلى أي صيغة صوتية أخرى.
تقوم الدالة WriteWavFile بإنشاء ملف WAV اعتماداً على المعلومات المخزنة في الـ Header باستخدام مكتبة NAudio وذلك لجميع قيم الـ Bit Depth المدعومة. أما في حالة استخدام عمق بت يساوي 8 بت، فيتم التعامل معها بشكل مختلف. السبب في ذلك أن العينات الصوتية داخل التطبيق تكون ممثلة بقيم Floating Point ضمن المجال [-1, 1]، بينما يعتمد تنسيق WAV ذي العمق 8 بت على تمثيل Unsigned PCM ضمن المجال [0, 255].
لذلك يتم تحويل كل عينة صوتية من المجال [-1, 1] إلى المجال [0, 255] قبل كتابتها إلى الملف. كما يتم أولاً حصر القيم ضمن المجال المسموح به لتجنب أي قيم غير صالحة، ثم تُحوّل إلى قيمة بايت واحدة (Byte) تمثل العينة داخل ملف WAV.
3. بعد إنشاء ملف WAV الوسيط يتم تحويله إلى الصيغة الأصلية التي كان عليها الملف قبل عملية الضغط. ويتم تحديد الصيغة المطلوبة من خلال قيمة OriginalFormat المخزنة مسبقاً داخل الـ Header.
تم دعم عمليات التحويل التالية:

- **التحويل إلى MP3** : يتم استخدام مكتبة NAudio.Lame لإنشاء ملف MP3 جديد من ملف WAV.
- **التحويل إلى FLAC** : يتم استخدام FFmpeg لتحويل ملف WAV إلى FLAC. وبما أن FLAC يستخدم ضغطاً بدون فقدان للبيانات (Lossless Compression)، فإن جودة الملف الناتج تبقى مطابقة لجودة ملف WAV الوسيط.
- **التحويل إلى AIFF** : يتم إنشاء ملف AIFF باستخدام AiffFileWriter مع الحفاظ على معدل العينات وعدد القنوات وعمق البت المطلوب.
- **التحويل إلى WMA** : يتم استخدام FFmpeg لتحويل الملف إلى صيغة WMA مع تحديد معدل البت المطلوب. تعتمد العملية على المرمز WMAV2 لتوليد الملف النهائي.

- **التحويل إلى AAC وM4A:** يتم استخدام FFmpeg مع رمز AAC لإنشاء الملفات بصيغتي AAC و M4A مع المحافظة على معدل البت المخزن أثناء عملية الضغط.
- **معالجة الصيغ غير المدعومة**
في حال كانت الصيغة المطلوبة غير مدعومة للتحويل، يتم الاحتفاظ بملف WAV الناتج وإرجاعه مباشرة باعتباره صيغة صوتية قياسية يمكن تشغيلها على معظم الأنظمة والبرامج.
- وأخيراً ثم حذف الملف الوسيط في حال تم إنشاء ملف نهائي بصيغة مختلفة عن ال WAV.

شرح تطبيق فك الضغط لكل خوارزمية

1. خوارزمية Delta Modulation

- تعتمد عملية فك الترميز على إعادة بناء الإشارة الصوتية انطلاقاً من قيمة متوقعة (Predicted Sample) لكل قناة صوتية. ويتم المرور على جميع البتات المخزنة بالترتيب نفسه الذي استخدم أثناء الضغط، حيث يتم استخراج كل بت من البايت المناسب باستخدام عمليات الإزاحة الثنائية (Bit Shifting) والـ Masking .
- بعد استخراج قيمة البت يتم تحديث القيمة المتوقعة وفق القواعد التالية:
- إذا كانت قيمة البت تساوي (1) تتم زيادة القيمة المتوقعة بمقدار StepSize.
 - إذا كانت قيمة البت تساوي (0) يتم إنقاص القيمة المتوقعة بالمقدار نفسه.
- بعد كل تحديث يتم تقييد القيمة ضمن المجال [-1,1] ثم تخزينها كعينة صوتية ضمن مصفوفة العينات الناتجة. وبهذه الطريقة يتم إعادة بناء الإشارة الصوتية تدريجياً اعتماداً على اتجاهات التغير المخزنة داخل الملف المضغوط، وهي العملية العكسية تماماً لآلية الضغط المستخدمة في خوارزمية Delta Modulation .

2. خوارزمية Adaptive Delta Modulation (ADM)

- تعتمد عملية فك الترميز في خوارزمية Adaptive Delta Modulation على إعادة بناء الإشارة الصوتية انطلاقاً من قيمة متوقعة (Predicted Sample) لكل قناة صوتية، مع إعادة تطبيق آلية التكيف نفسها التي استخدمت أثناء عملية الضغط. يتم أولاً قراءة القيم المخزنة في ال-Header مثل قيمة الخطوة الابتدائية (Initial Step Size) ، والحدود الدنيا والعليا لحجم الخطوة، ومعاملات الزيادة والتخفيض، بالإضافة إلى طول سجل البتات المستخدم في عملية التكيف.
- بعد ذلك يتم المرور على جميع البتات المخزنة بالترتيب نفسه الذي استخدم أثناء الضغط، حيث يتم استخراج كل بت من البايت المناسب باستخدام عمليات الإزاحة الثنائية (Bit Shifting) والـ Masking .
- بعد استخراج قيمة البت يتم تحديث القيمة المتوقعة وفق القواعد التالية:
- إذا كانت قيمة البت تساوي (1) تتم زيادة القيمة المتوقعة بمقدار Step Size الحالية.
 - إذا كانت قيمة البت تساوي (0) يتم إنقاص القيمة المتوقعة بالمقدار نفسه.

بعد كل تحديث يتم تقييد القيمة المتوقعة ضمن المجال [-1,1] ثم تخزينها كعينة صوتية ضمن مصفوفة العينات الناتجة.

بعد ذلك يتم تحديث سجل البتات (Bit History) الخاص بالقناة الحالية، والذي يُستخدم لتحديد كيفية تعديل حجم الخطوة:

- إذا احتوى السجل على عدد متتالي من البتات المتشابهة يساوي القيمة المحددة في BitHistoryLength ، فهذا يدل على أن الإشارة تتغير باستمرار في الاتجاه نفسه، وعندها يتم زيادة حجم الخطوة بضربها بالمعامل StepMultiplier.

- أما إذا لم تكن جميع البتات متشابهة، فيتم تقليل حجم الخطوة باستخدام المعامل StepDecay.

بعد كل عملية تعديل يتم التأكد من بقاء قيمة Step Size ضمن المجال المحدد بين MinStepSize و MaxStepSize .

3. خوارزمية (DPCM) Differential Pulse Code Modulation

تعتمد عملية فك الترميز في خوارزمية DPCM على إعادة بناء العينات الصوتية انطلاقاً من أول عينة أصلية لكل قناة صوتية والفروق المكممة المخزنة داخل الملف المضغوط.

يتم أولاً قراءة المعلومات المخزنة في ال-Header ، والتي تتضمن عدد العينات، وعدد القنوات، وعدد مستويات التكميم، وأصغر وأكبر فرق تم استخدامهما أثناء الضغط، بالإضافة إلى أول عينة لكل قناة صوتية.

بعد ذلك يتم استخراج القيم المكممة المخزنة داخل الملف باستخدام عملية Unpack Bits ، والتي تقوم بعكس عملية Bit Packing المستخدمة أثناء الضغط. حيث يتم قراءة عدد البتات المناسب لكل قيمة وفق عدد مستويات التكميم المستخدمة، ثم إعادة تكوين القيم المكممة الأصلية.

بعد الحصول على القيم المكممة تبدأ عملية إعادة بناء الإشارة وفق الخطوات التالية:

- 1- يتم إنشاء مصفوفة مستقلة لكل قناة صوتية.
- 2- يتم وضع أول عينة لكل قناة باستخدام القيم المخزنة في المتغير InitialSamples.
- 3- يتم تحويل كل قيمة مكممة إلى قيمة مطبوعة (Normalized Value) ضمن المجال [0,1]
- 4- يتم إعادة حساب الفرق الحقيقي (Difference) باستخدام أصغر وأكبر فرق مخزينين في ال-Header.
- 5- يتم الحصول على العينة الحالية من خلال إضافة الفرق المستعاد إلى العينة السابقة وفق العلاقة:

$$\text{Current Sample} = \text{Previous Sample} + \text{Difference}$$

- 6- تتكرر العملية لجميع العينات حتى تتم إعادة بناء الإشارة الكاملة لكل قناة صوتية.

بعد الانتهاء من إعادة بناء جميع القنوات يتم دمجها مجدداً (Interleaving) لإعادة ترتيب العينات بالشكل القياسي المستخدم في الملفات الصوتية متعددة القنوات.

وفي النهاية يتم تقييد جميع العينات ضمن المجال [1,1] للتأكد من عدم تجاوز الحدود المسموح بها للإشارة الصوتية.

الطلب السابع: مراقبة الأداء أثناء التنفيذ

لتحقيق إمكانية مراقبة أداء عملية الضغط بشكل لحظي (Real-Time) أثناء التنفيذ تم إنشاء حدث (Event) من النوع `Action<CompressionProgress>` باسم `ProgressChanged` داخل الصف الأساسي للخوارزميات. يتيح هذا الحدث إرسال تحديثات دورية من الخوارزمية إلى واجهة المستخدم كلما توفرت معلومات جديدة عن حالة التنفيذ. كما تم إنشاء الصف `CompressionProgress` ليحمل المعلومات التي تحتاجها الواجهة لتحديث عناصر المراقبة، وتشمل:

- **Percentage**: نسبة الإنجاز الحالية من عملية الضغط.

- **ProcessingSpeed**: سرعة المعالجة أثناء التنفيذ.

- **CompressionRatio**: نسبة الضغط المحققة حتى تلك اللحظة.

- **ElapsedMs**: الزمن المنقضي منذ بدء العملية.

أثناء تنفيذ عملية الضغط تقوم كل خوارزمية باستدعاء الدالة `ReportProgress` بشكل دوري، والتي بدورها تقوم بإطلاق الحدث `ProgressChanged` وتمرير كائن يحتوي على أحدث قيم الأداء. ولضمان بقاء واجهة المستخدم مستجيبة أثناء الضغط، تم تنفيذ عملية الضغط داخل `Thread` منفصل باستخدام `Task.Run`، وبهذه الطريقة تستمر الخوارزمية بالعمل في الخلفية بينما تبقى الواجهة قادرة على استقبال التحديثات والتفاعل مع المستخدم دون تجمد. وعند استقبال الحدث في الواجهة يتم استخدام `Invoke` لتنفيذ تحديثات عناصر الواجهة بشكل آمن على `UI Thread`، حيث يتم تحديث:

- شريط التقدم (Progress Bar).

- النسبة المئوية الحالية للإنجاز.

- مخطط سرعة المعالجة (Processing Speed).

- مخطط نسبة الضغط (Compression Ratio).

الطلب الثامن: إلغاء عملية الضغط أثناء التنفيذ

كما ذكرنا في الطلب السابق، يتم تنفيذ عملية الضغط داخل `Thread` منفصل باستخدام `Task.Run`، مما يسمح للخوارزمية بالعمل في الخلفية مع استمرار تحديث واجهة المستخدم وعرض معلومات الأداء بشكل لحظي.

ولتحقيق الإلغاء تم الاعتماد على CancellationToken. عند بدء كل عملية ضغط يتم إنشاء CancellationTokenSource جديد، ثم يتم تمرير الـ Token إلى الخوارزمية المستخدمة من خلال الدالة SetCancellationToken.

أثناء تنفيذ عملية الضغط تقوم الخوارزمية بالتحقق من حالة الـ Token بشكل دوري ضمن الحلقة الرئيسية للمعالجة. وقد تم تنفيذ هذا الفحص عند كل فترة تحديث للأداء (reportInterval) وذلك لتجنب إجراء الفحص عند كل عينة صوتية وتحقيق توازن بين سرعة الاستجابة وكفاءة التنفيذ.

عند طلب الإلغاء من خلال الزر Cancel يتم استدعاء الدالة:

CancellationToken.ThrowIfCancellationRequested();

تقوم هذه الدالة برفع Exception من النوع OperationCanceledException بشكل تلقائي، مما يؤدي إلى إيقاف تنفيذ الخوارزمية والخروج من عملية الضغط مباشرة. بعد ذلك ينتقل التنفيذ إلى catch الموجودة في واجهة المستخدم، حيث تتم معالجة الإلغاء بالشكل المناسب، ويتضمن ذلك:

- إيقاف عملية الضغط بشكل كامل.
- حذف الملف المضغوط في حال كان قد تم إنشاء جزء منه قبل الإلغاء.
- إعادة تعيين شريط التقدم إلى الصفر.
- تنظيف الرسوم البيانية الخاصة بالأداء.
- إعادة تفعيل عناصر الواجهة وإظهار رسالة تفيد بإلغاء العملية.

الطلب التاسع: إعادة ضبط القيم الأصلية للملف الصوتي

لتحقيق ذلك تم الاحتفاظ بمسار الملف الصوتي الأصلي مباشرة بعد تحميله إلى التطبيق ضمن متغير مخصص لا يتم تعديله أثناء تنفيذ عمليات الضغط أو فك الضغط أو تغيير الإعدادات. وبهذا يبقى مرجع دائم للملف الأصلي متاحاً طوال فترة عمل التطبيق. عند اختيار المستخدم لإعادة الضبط، يتم استخدام هذا المسار لإعادة تحميل الملف الأصلي من جديد، ومن ثم إعادة عرض خصائصه الأصلية وإعادة إنشاء الموجة الصوتية الخاصة به، بالإضافة إلى إعادة تهيئة جميع القيم والإعدادات المرتبطة بالملف.

الطلب العاشر: عرض تقرير بعد انتهاء عملية الضغط

تم تحقيق هذا الطلب من خلال إنشاء الدالة GenerateReport التي يتم استدعاؤها مباشرة بعد انتهاء عملية الضغط بنجاح. تقوم هذه الدالة بجمع المعلومات المطلوبة من الملف الأصلي والملف المضغوط، ثم تخزينها ضمن المتغير reportText ليتم عرضها لاحقاً عند ضغط المستخدم على زر عرض التقرير. يحتوي التقرير على المعلومات التالية:

- **Original Size**: حجم الملف الأصلي قبل الضغط، ويتم الحصول عليه من خلال الخاصية Length الخاصة بالكائن FileInfo للملف الأصلي.
- **Compressed Size**: حجم الملف الناتج بعد الضغط، ويتم الحصول عليه بالطريقة نفسها من الملف المضغوط.
- **Size Reduction**: نسبة تقليل الحجم الناتجة عن عملية الضغط، ويتم حسابها وفق العلاقة:

$$\text{Reduction \%} = (1 - \text{CompressedSize} / \text{OriginalSize}) \times 100$$
- **Compression Time**: الزمن اللازم لإتمام عملية الضغط، ويتم الحصول عليه من قيمة elapsedMs المقاسة باستخدام Stopwatch أثناء تنفيذ الخوارزمية.
- **Algorithm**: اسم خوارزمية الضغط المستخدمة، ويتم الحصول عليه من الإعدادات الحالية المخزنة ضمن CompressionSettings.
- **Sample Rate**: معدل العينات المستخدم أثناء الترميز، ويتم استخراجه من الـ Header الخاص بالملف المضغوط.
- **Encoded Bit Rate**: معدل البت الناتج بعد الضغط، ويتم استخراجه من الـ Header الذي تم إنشاؤه أثناء عملية الضغط.
- **Bit Depth**: عمق البت المستخدم بعد المعالجة المسبقة، ويتم قراءته من الـ Header.
- **Channels**: عدد القنوات الصوتية المستخدمة في الملف المضغوط، ويتم قراءته من الـ Header.

بالإضافة إلى ذلك يتم عرض الإعدادات الخاصة بالخوارزمية المستخدمة.

1. خوارزمية Delta Modulation يتم استخراج وعرض:

- Step Size: قيمة الخطوة المستخدمة أثناء الترميز، ويتم قراءتها من القسم الخاص بالخوارزمية داخل الـ Header.

2. خوارزمية Adaptive Delta Modulation (ADM):

- يتم استخراج وعرض مجموعة معاملات الخوارزمية المخزنة داخل الـ Header أثناء عملية الضغط، وهي المعاملات المسؤولة عن التحكم بآلية التكيف الديناميكي لقيمة الخطوة أثناء الترميز، وتشمل:
 - Initial Step Size: قيمة الخطوة الابتدائية التي تبدأ بها الخوارزمية قبل بدء عملية التكيف.
 - Min Step Size: أصغر قيمة مسموح بها للخطوة أثناء عملية الترميز، وتستخدم لمنع انخفاض حجم الخطوة إلى قيم صغيرة جداً تؤثر على قدرة الخوارزمية على تتبع الإشارة.
 - Max Step Size: أكبر قيمة مسموح بها للخطوة، وتستخدم لمنع تضخم قيمة الخطوة بشكل يؤدي إلى فقدان الدقة.
 - Step Multiplier: معامل زيادة حجم الخطوة عند اكتشاف اتجاه تغير مستمر في الإشارة، مما يساعد الخوارزمية على تتبع التغيرات السريعة بكفاءة أكبر.

- Step Decay: معامل تقليل حجم الخطوة عند عدم وجود اتجاه ثابت للتغير، مما يسمح بتحسين دقة تمثيل الإشارة في المناطق الهادئة أو قليلة التغير.
- Bit History Length: عدد البتات السابقة التي يتم الاحتفاظ بها لمراقبة نمط التغير في الإشارة واتخاذ قرار زيادة أو تقليل حجم الخطوة.

نُقرأ هذه القيم من القسم الخاص بخوارزمية ADM داخل الـ Header وتُعرض ضمن التقرير بهدف توضيح الإعدادات التي استُخدمت أثناء عملية الضغط.

3. خوارزمية (DPCM) Differential Pulse Code Modulation :

تعتمد المعلومات المعروضة في التقرير لهذه الخوارزمية على إعدادات التكميم المستخدمة أثناء الترميز، ويتم استخراجها من القسم الخاص بالخوارزمية داخل الـ Header.

- Quantization Levels: عدد مستويات التكميم المستخدمة لتحويل فروقات العينات إلى قيم رقمية محددة قبل تخزينها. تؤدي زيادة عدد المستويات إلى تمثيل أدق للفروقات وتحسين جودة الإشارة المعاد بناؤها، في حين تسمح القيم الأصغر بتحقيق ضغط أعلى على حساب الدقة.

يتم عرض هذه القيمة ضمن التقرير لأنها تمثل أحد أهم العوامل المؤثرة على التوازن بين نسبة الضغط وجودة الصوت الناتجة في خوارزمية DPCM.

الطلب الحادي عشر: حفظ الملف المضغوط والملف بعد الضغط

لتحقيق ذلك تم تعريف متغيرين لتخزين مسارات الملفات الناتجة، أحدهما للاحتفاظ بمسار الملف المضغوط الناتج عن عملية الضغط، والآخر للاحتفاظ بمسار الملف الناتج عن عملية فك الضغط.

عند اختيار المستخدم حفظ الملف، يتم عرض نافذة من نوع SaveFileDialog تسمح له بتحديد اسم الملف وموقع الحفظ المطلوب. وبعد اختيار المسار المناسب يتم نسخ الملف الناتج من موقعه المؤقت إلى الموقع الذي حدده المستخدم.

رابط المشروع على الـ Git Hub

<https://github.com/TH26-m10/AudioCompression>